

Web Technology

Complete MCQ-Oriented Notes

IBPS SO (IT OFFICER)
COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE
CONCEPT-FOCUSED LEARNING
MCQ-ORIENTED APPROACH
QUICK REVISION READY

COMPILED BY
Asabhya Maanav

2026 EDITION
VERSION 1.0

Table of Contents

Web Technology — Complete Notes	4
1. Web Fundamentals	4
1.1 Internet, WWW, Browser, and Server	4
1.2 HTTP / HTTPS Request-Response Cycle	4
1.3 URL Structure and DNS	5
1.4 Client-Side vs Server-Side; Static vs Dynamic	5
2. HTML — HyperText Markup Language	6
2.1 HTML Basics	6
2.2 HTML Lists	7
2.3 HTML Tables	7
2.4 Images and Multimedia	7
2.5 Frames	7
2.6 HTML Forms	7
2.7 Semantic and HTML5 Elements	8
3. CSS — Cascading Style Sheets	8
3.1 Introduction and Syntax	8
3.2 Types of CSS	8
3.3 Selectors	9
3.4 CSS Properties	9
3.5 The Box Model	9
3.6 Positioning, Display, Float	9
3.7 Cascading and Specificity	10
4. DHTML — Dynamic HTML	10
5. Client-Side Scripting	10
5.1 JavaScript	10
5.2 AJAX	11
5.3 VBScript	12
6. Java Beans and EJB	12
6.1 Java Beans	12
6.2 EJB (Enterprise Java Beans)	13
7. Server-Side Programming	13
7.1 ASP — Active Server Pages	13
7.2 JSP — Java Server Pages	14
8. Servlets	15
8.1 Introduction	15
8.2 Servlet Packages and API	15

8.3 Servlet Lifecycle	15
8.4 Request Handling and Session Tracking	16
8.5 Servlets vs JSP	16
8.6 JDBC (Database Programming)	16
9. Distributed Component Technologies	16
9.1 COM (Component Object Model)	16
9.2 DCOM (Distributed COM)	16
9.3 CORBA	17
9.4 Comparison of COM / DCOM / CORBA	17
10. PHP — Hypertext Preprocessor	17
10.1 Introduction and Setup	17
10.2 Syntax	17
10.3 Variables	17
10.4 Strings and String Functions	17
10.5 Operators	18
10.6 Control Structures	18
10.7 Arrays	18
10.8 Functions	18
10.9 Form Handling (GET/POST)	18
10.10 Mail	18
10.11 File Upload and File Handling	18
10.12 Sessions and Cookies	19
10.13 Error and Exception Handling	19
10.14 Filter (Validation and Sanitization)	19
10.15 PHP-ODBC and PHP-MySQL	19
11. Quick Revision — Exam Pointers	19

Web Technology — Complete Notes

This guide deconstructs the IBPS SO (IT Officer) Web Technology syllabus to the atomic level: every tag, selector, parser, implicit object, lifecycle method, port, and examiner trap. Concepts build progressively from beginner definitions to advanced server-side architecture. Web Technology is split into **client-side** (runs in the browser) and **server-side** (runs on the web server) technologies, and this entire document mirrors that division.

1. Web Fundamentals

1.1 Internet, WWW, Browser, and Server

1.1.1 Core Definitions

The **Internet** is a global network of interconnected networks using the **TCP/IP** protocol suite; it is the **infrastructure**.

The **World Wide Web (WWW)** is a **service** that runs *on top of* the Internet — a collection of **hyperlinked documents** identified by URLs and accessed via **HTTP**. It was invented by **Tim Berners-Lee** in **1989** at **CERN**.

A **web browser** is a **client** program that requests, fetches, and **renders** web pages (e.g., **Chrome**, **Firefox**, **Safari**, **Edge**).

A **web server** is software/hardware that **stores** web resources and **responds** to HTTP requests (e.g., **Apache**, **Nginx**, **Microsoft IIS**).

MCQ Trap: The **Internet** is the hardware/network infrastructure; the **WWW** is just one service on it (others are email, FTP). They are NOT synonyms — examiners love to swap these.

1.1.2 Browser Rendering Engines

Browser	Rendering (Layout) Engine	JavaScript Engine
Chrome	Blink	V8
Firefox	Gecko	SpiderMonkey
Safari	WebKit	JavaScriptCore
Old IE	Trident	Chakra/JSScript

1.2 HTTP / HTTPS Request-Response Cycle

1.2.1 HTTP Basics

HTTP (HyperText Transfer Protocol) is the **application-layer**, **stateless** protocol that governs communication between browser and server. Default port is **80**.

HTTPS is **HTTP over TLS/SSL** — encrypted; default port is **443**.

HTTP is stateless: each request is independent; the server remembers nothing between requests. State is added externally via **cookies**, **sessions**, and **URL rewriting**.



1.2.2 HTTP Methods

Method	Purpose	Body?	Idempotent?
GET	Retrieve a resource	No	Yes
POST	Submit/create data	Yes	No
PUT	Replace/update a resource	Yes	Yes
DELETE	Remove a resource	No	Yes
HEAD	Headers only (no body)	No	Yes
PATCH	Partial update	Yes	No

1.2.3 HTTP Status Codes

- **1xx** — Informational (e.g., **100 Continue**).
- **2xx** — Success: **200 OK**, **201 Created**, **204 No Content**.
- **3xx** — Redirection: **301 Moved Permanently**, **302 Found**, **304 Not Modified**.
- **4xx** — Client error: **400 Bad Request**, **401 Unauthorized**, **403 Forbidden**, **404 Not Found**.
- **5xx** — Server error: **500 Internal Server Error**, **502 Bad Gateway**, **503 Service Unavailable**.

Memory Trick: "1-Info, 2-Yes!, 3-Go elsewhere, 4-You erred, 5-I erred". **404** = client asked for missing page; **500** = server crashed.

1.3 URL Structure and DNS

A **URL (Uniform Resource Locator)** addresses a resource. General form:

```

scheme://host:port/path?query#fragment
https://www.example.com:443/products/item?id=42#reviews
  |         |         |         |         |         |
  scheme  host   port   path   query  fragment

```

DNS (Domain Name System) translates human-readable **domain names** into machine **IP addresses**. It runs on port **53** and is often called the "**phonebook of the Internet**."

MCQ Trap: **URL** locates a resource (includes how to get it — the scheme); **URI** identifies it; **URN** names it. A URL is a *type* of URI.

1.4 Client-Side vs Server-Side; Static vs Dynamic

Aspect	Client-Side	Server-Side
Runs on	The browser	The web server
Examples	HTML, CSS, JavaScript, VBScript	PHP, JSP, ASP, Servlets
Code visible?	Yes (View Source)	No (hidden)
Use	UI, validation, interactivity	DB access, business logic

- A **static web page** has fixed content (pure HTML); every user sees the same thing.
- A **dynamic web page** is generated on the fly (server-side or client-side); content changes per user/request/database.

Memory Trick: "**Client = Cosmetics, Server = Substance**." Validation can be client-side for speed but **MUST** be repeated server-side for security.

2. HTML — HyperText Markup Language

2.1 HTML Basics

2.1.1 Nature of HTML

HTML is a **markup language** (NOT a programming language) used to **structure** web content. It is **not case-sensitive** (but lowercase is the convention). The current standard is **HTML5**.

A **tag** is a keyword in angle brackets like `<p>`; an **element** is the complete unit `<p>...</p>` (start tag + content + end tag); an **attribute** provides extra info inside the start tag as `name="value"`.

```
<p class="intro">Hello</p>
|      |      |      |
start  |      |      |
tag    (name=value)  tag
<----- element ----->
```

2.1.2 Document Structure

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Heading</h1>
    <p>Paragraph</p>
  </body>
</html>
```

- `<!DOCTYPE html>` declares the document is **HTML5** (it is **not** a tag).
- `<head>` holds **metadata** (title, meta, links to CSS) — not displayed in the page body.
- `<body>` holds the **visible** content.

2.1.3 Empty (Void) Elements

Elements with **no content and no closing tag**: `<br>` (line break), `<hr>` (horizontal rule), `<img>`, `<input>`, `<meta>`, `<link>`.

MCQ Trap: `<br>`, `<hr>`, `<img>`, `<input>` are **void/empty** elements — they have NO closing tag. A frequent trick is asking which tag needs no end tag.

2.1.4 Headings, Paragraphs, Text Formatting

- Headings: `<h1>` (largest/most important) down to `<h6>` (smallest) — **6 levels**.
- Paragraph: `<p>`; line break `<br>`; horizontal rule `<hr>`.
- Formatting: `<b>` bold, `<strong>` important, `<i>` italic, `<em>` emphasis, `<u>` underline, `<mark>` highlight, `<sub>` subscript, `<sup>` superscript, `<del>` strikethrough, `<ins>` inserted.

MCQ Trap: `<b>`/`<i>` are **visual/presentational**; `<strong>`/`<em>` are **semantic** (meaning). They look identical but differ in purpose — screen readers emphasize `<strong>`/`<em>`.

2.1.5 Hyperlinks and Anchors

The anchor tag `<a href="url">text</a>` creates a hyperlink. The **target** attribute controls where it opens:

- `_self` — same tab (default).
- `_blank` — new tab/window.
- `_parent` / `_top` — frame navigation.

2.2 HTML Lists

There are **3 types** of HTML lists.

List	Tag	Items	Notes
Unordered			Bullets: disc, circle, square
Ordered			Numbered; type=1/A/a/I/i
Definition	<dl>	<dt>+<dd>	Term + description

- **** supports **type (1, A, a, I, i)** and a **start** attribute.
- **<dl>** uses **<dt>** (definition term) and **<dd>** (definition description).

2.3 HTML Tables

A table is built with **<table>**; each row is **<tr>**; a header cell is **<th>** (bold + centered); a data cell is **<td>**.

- Structural tags: **<caption>** (title), **<thead>**, **<tbody>**, **<tfoot>**, **<colgroup>**.
- **Spanning**: **colspan="n"** merges cells **horizontally** (across columns); **rowspan="n"** merges **vertically** (across rows).
- Attributes (HTML4, now done via **CSS**): **border**, **cellspacing** (between cells), **cellpadding** (inside cells).

```

+-----+
|           <caption>           |
+-----+-----+-----+
| <th> | <th> | <th> | <thead>
+-----+-----+-----+
| <td> | <td colspan=2> | <tbody>
+-----+-----+-----+

```

MCQ Trap: **colspan = columns** (horizontal merge); **rowspan = rows** (vertical merge). **cellspacing** is the gap **between** cells; **cellpadding** is space **inside** a cell.

2.4 Images and Multimedia

- **** embeds an image; it is a **void** element. **alt** provides **alternative text** for accessibility/broken images.
- **<audio controls>** and **<video controls>** (HTML5) embed media; the **<source>** tag specifies files; **controls** shows play/pause.

2.5 Frames

- **<frameset>** divides the window using **rows/cols** attributes and **replaces <body>**; each pane is a **<frame src name>**. **<noframes>** is fallback content.
- **<iframe src="...">** is an **inline frame** — embeds another document inside the current page (still valid in HTML5).

MCQ Trap: **<frameset>** and **<frame>** are **removed/deprecated in HTML5**, but **<iframe>** remains **valid**. A **<frameset>** document has **NO <body>**.

2.6 HTML Forms

2.6.1 Form Element and Attributes

<form> collects user input. Key attributes:

- **action** — URL that processes the submitted data.
- **method** — **GET** or **POST**.
- **enctype** — encoding type (matters for file upload).

- **name, target, autocomplete, novalidate.**

2.6.2 GET vs POST

Feature	GET	POST
Data location	In URL (query string)	In request body
Visible in URL?	Yes	No
Size limit	Limited (~2048 chars)	Practically unlimited
Bookmarkable/Cached	Yes	No
Idempotent	Yes	No
Use case	Search, retrieval	Login, upload, sensitive data

MCQ Trap: **GET** appends data to the URL (visible, cached, size-limited); **POST** hides it in the body (secure-ish, large). The **default** method of a form is **GET**.

2.6.3 Input Types and Controls

- Classic `<input type>`: **text, password, radio, checkbox, submit, reset, button, file, hidden, image.**
- HTML5 additions: **email, number, date, range, color, tel, url, search, time.**
- Other controls: `<select>+<option>` (dropdown), `<textarea>` (multi-line), `<label>`, `<fieldset>+<legend>`, `<datalist>`, `<button>`.

MCQ Trap: **Radio buttons** with the SAME **name** allow only **one** selection; **checkboxes** allow **multiple**. `<input type="file">` requires `enctype="multipart/form-data"` and `method="POST"`.

2.7 Semantic and HTML5 Elements

Semantic tags describe **meaning**, not just appearance: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>`, `<figure>`, `<figcaption>`, `<details>`, `<summary>`. They improve **SEO** and **accessibility** versus generic `<div>`.

3. CSS — Cascading Style Sheets

3.1 Introduction and Syntax

CSS describes the **presentation** (look) of HTML — separating **content** from **style**. It is maintained by the **W3C**. Syntax:

```
selector {
  property: value;
  color: red;
}
```

A rule = **selector** + **declaration block**; a declaration = **property : value**;

3.2 Types of CSS

Type	Where	Syntax	Specificity
Inline	In the tag	<code>style="color:red"</code>	Highest
Internal	In <code><head></code>	<code><style>...</style></code>	Medium
External	Separate <code>.css</code> file	<code><link rel="stylesheet"></code>	Lowest, best reuse

MCQ Trap: **External CSS** is best for maintainability (one file, many pages). **Inline CSS** has the **highest priority** among the three placements but is hardest to maintain.

3.3 Selectors

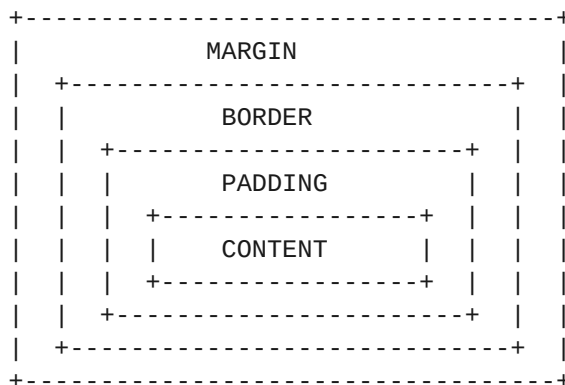
Selector	Symbol	Selects
Universal	*	All elements
Element/Type	p	All <p>
Class	.name	Elements with that class
ID	#name	The one element with that id
Group	a, b	Multiple selectors
Descendant	div p	<p> inside <div>
Child	div > p	Direct child only
Adjacent sibling	h1 + p	Next sibling
Pseudo-class	a:hover	State-based
Pseudo-element	p::before	Part of element

3.4 CSS Properties

- Text/Font: **color**, **font-family**, **font-size**, **font-weight**, **font-style**, **text-align**, **text-decoration**, **line-height**.
- Background: **background-color**, **background-image**, **background-repeat**, **background-position**.
- Border: **border**, **border-width**, **border-style**, **border-color**, **border-radius**.

3.5 The Box Model

Every element is a rectangular box with **4 layers** (inside → out): **content** → **padding** → **border** → **margin**.



- Content** — the actual text/image.
- Padding** — transparent space **inside** the border (around content).
- Border** — line around the padding.
- Margin** — transparent space **outside** the border (between elements).
- Total width (default **content-box**) = **width** + 2×padding + 2×border + 2×margin.
- box-sizing: border-box** makes **width include** padding and border.

MCQ Trap: Padding is inside the border (between content and border); **Margin** is outside the border (between elements). Margins can **collapse**; padding never collapses.

3.6 Positioning, Display, Float

- position: static** (default, normal flow), **relative** (offset from normal), **absolute** (relative to nearest positioned ancestor), **fixed** (relative to viewport), **sticky** (hybrid).

- **display:** **block** (full width, new line), **inline** (no width/height, flows in text), **inline-block**, **none** (removed), **flex**, **grid**.
- **float:** **left/right** removes element from normal flow; **clear** stops floating.

3.7 Cascading and Specificity

When rules conflict, the **cascade** decides via: **importance** → **specificity** → **source order**.

Specificity weight (a,b,c,d):

- Inline style = **1000**.
- ID selector = **100**.
- Class / attribute / pseudo-class = **10**.
- Element / pseudo-element = **1**.
- Universal * = **0**.

!important overrides everything (except another later **!important**).

MCQ Trap: Order of priority: **!important** > **inline** > **ID** > **class** > **element**. If two rules tie on specificity, the **last declared** wins (source order).

4. DHTML — Dynamic HTML

DHTML is **NOT** a language, technology, or W3C standard. It is a **combination/term** for building dynamic, interactive pages **on the client side** without reloading from the server.

DHTML = **HTML** (content/structure) + **CSS** (presentation) + **JavaScript** (behavior) + **DOM** (the glue that lets JS access/modify HTML & CSS).

```

      DHTML
     /  |  \  \
    /   |   \  \
   HTML CSS JS  DOM
   (content)(style)(logic)(binds all three)
  
```

It is **event-based**: content changes in response to events (clicks, mouse moves, timers) without a server round-trip.

MCQ Trap: DHTML is **not** a single technology — it is the **integration** of HTML + CSS + JavaScript + DOM. There is no "DHTML compiler" or "DHTML file extension."

5. Client-Side Scripting

5.1 JavaScript

5.1.1 Introduction and Embedding

JavaScript (JS) is a **client-side**, **interpreted**, **dynamically (loosely) typed**, **case-sensitive**, object-based scripting language. It was created by **Brendan Eich** at **Netscape** in **1995**. Its standard is **ECMAScript (ECMA-262)**.

MCQ Trap: JavaScript is **NOT** related to Java — only the name is similar (a marketing decision). JS is interpreted and loosely typed; Java is compiled and strongly typed.

Embedding methods:

- Internal: inside `<script> ... </script>` (in `<head>` or before `</body>`).
- External: `<script src="app.js"></script>`.
- Inline: `<button onclick="fn()">`.

5.1.2 Variables, Data Types, Operators

- Declarations: **var** (function-scoped, hoisted), **let** (block-scoped), **const** (block-scoped, constant).

- **Primitive types (7):** **Number**, **String**, **Boolean**, **Undefined**, **Null**, **Symbol** (ES6), **BigInt** (ES2020). Plus the non-primitive **Object** (includes Array, Function).
- `typeof null` returns **"object"** — a famous historical bug.
- Operators: arithmetic (+ - * / %), comparison (==, ===, !=, !==), logical (&&, ||, !), ternary ? :.

MCQ Trap: `==` compares **value only** (with type coercion); `===` is **strict** — compares **value AND type** with no coercion. So `5 == "5"` is **true** but `5 === "5"` is **false**.

5.1.3 Statements (Control Structures and Loops)

- Conditionals: `if`, `if-else`, `else if`, `switch`.
- Loops: `for`, `while`, `do-while`, `for-in` (object **keys**), `for-of` (iterable **values**).
- Jump: `break`, `continue`.

5.1.4 Functions

Defined via declaration `function f(){}` , expression `var f = function(){}` , or arrow `const f = () => {}` . Function **declarations are hoisted**; expressions are not. Functions are **first-class objects** in JS.

5.1.5 Built-in Objects

Core built-ins: **Math**, **Date**, **String**, **Array**, **Number**, **Boolean**, **RegExp**, **JSON**, **Object**. The **BOM (Browser Object Model)** adds: `window`, `navigator`, `screen`, `location`, `history`.

5.1.6 Documents (DOM Manipulation)

The **DOM (Document Object Model)** represents the page as a tree of nodes that JS can read/modify. Common methods:

- Selecting: `document.getElementById()`, `getElementsByClassName()`, `getElementsByTagName()`, `querySelector()`, `querySelectorAll()`.
- Modifying: `createElement()`, `appendChild()`, `removeChild()`, `.innerHTML`, `.textContent`, `setAttribute()`.

5.1.7 Forms Handling and Validation

JS accesses forms via `document.forms[]` and `form.elements[]`. **Client-side validation** checks input before submission; returning **false** from `onsubmit` **cancels** submission. Dialog helpers: `alert()`, `prompt()`, `confirm()`.

MCQ Trap: Client-side validation improves UX/speed but is **NOT secure** — it can be bypassed (JS disabled). Always re-validate on the **server**.

5.1.8 Events and Event Handlers

Common events: `onclick`, `ondblclick`, `onload`, `onunload`, `onmouseover`, `onmouseout`, `onfocus`, `onblur`, `onchange`, `onsubmit`, `onkeydown`, `onkeyup`, `onkeypress`. Modern handler: `addEventListener("event", fn)`.

5.2 AJAX

5.2.1 Introduction

AJAX = Asynchronous JavaScript And XML. It is a **technique** (not a single technology) for exchanging data with the server **asynchronously** so that **part of a page updates without a full reload**. The term was coined by **Jesse James Garrett** in **2005**.

5.2.2 XMLHttpRequest Object

The heart of AJAX is the **XMLHttpRequest (XHR)** object.

- `open(method, url, async)` — configure the request.

- `send()` — dispatch it.
- Properties: `readyState`, `status`, `responseText`, `responseXML`.
- Handler: `onreadystatechange`.

The `readyState` has **5 values (0–4)**:

Value	State	Meaning
0	UNSENT	<code>open()</code> not called
1	OPENED	<code>open()</code> called
2	HEADERS_RECEIVED	<code>send()</code> called, headers in
3	LOADING	downloading response body
4	DONE	operation complete

MCQ Trap: A successful AJAX response is detected when `readyState == 4 AND status == 200`. `readyState 4` alone only means "finished" — it could still be a 404.

5.2.3 AJAX with JSON

Modern AJAX usually transfers **JSON** (not XML) — sometimes called **AJAJ**. The **`fetch()` API** and Promises are the modern replacement for raw XHR. Advantages: speed, less bandwidth, smoother UX. Disadvantages: **SEO** difficulties, broken **back button**, JavaScript dependency.

5.3 VBScript

VBScript (Visual Basic Scripting Edition) is a **Microsoft** scripting language based on **Visual Basic**. It is **case-insensitive** and works **client-side only in Internet Explorer** (and server-side in **ASP** / Windows Script Host). It is now **deprecated**.

Feature	JavaScript	VBScript
Vendor	Netscape/ECMA	Microsoft
Case sensitivity	Case-sensitive	Case-insensitive
Browser support	All browsers	IE only
Syntax style	C / Java-like	BASIC-like

MCQ Trap: VBScript runs client-side **only in Internet Explorer**, so it is **not cross-browser** — a key reason JavaScript won the client-side war.

6. Java Beans and EJB

6.1 Java Beans

6.1.1 Introduction and Conventions

A **JavaBean** is a **reusable software component** that can be manipulated **visually** in a builder tool. To qualify, a class must follow conventions:

- Be a **public** class with a **public no-argument constructor**.
- Expose **properties** through **getter/setter** methods (`getX()`/`setX()`, and `isX()` for boolean).
- Implement **Serializable** (for persistence).
- Support **events** via listeners.

6.1.2 Advantages

- **Reusability** across applications and platforms.
- **Portability** (pure Java — "write once, run anywhere").
- Supports **introspection**, **customization**, **persistence**, and **events**.

6.1.3 Properties

There are **4 types** of bean properties.

Property	Description
Simple	A single value (one getter/setter)
Indexed	An array of values (element + array access)
Bound	Notifies listeners on change via PropertyChangeListener
Constrained	Listeners can veto a change via VetoableChangeListener

MCQ Trap: A **bound** property **announces** a change (**PropertyChangeListener**); a **constrained** property lets a listener **reject/veto** it (**VetoableChangeListener**, throws **PropertyVetoException**). Don't swap these.

6.1.4 BDK and Java Beans API

- **BDK (Bean Development Kit)** is Sun's tool to create/test beans; its test container was the **BeanBox**.
- The Beans API lives in the **java.beans** package: **BeanInfo**, **Introspector**, **PropertyDescriptor**, **PropertyEditor**, **Customizer**.

6.1.5 Introspection, Customization, Persistence

- **Introspection** — analyzing a bean's properties, methods, and events automatically (via **reflection** + **BeanInfo**, using the **Introspector** class).
- **Customization** — changing a bean's appearance/behavior at design time using property editors/customizers.
- **Persistence** — saving and restoring a bean's state through **serialization**.

6.2 EJB (Enterprise Java Beans)

EJB is a **server-side** component model for building **distributed**, **transactional**, **secure** enterprise applications. EJBs run inside an **EJB container** (part of **Java EE / Jakarta EE**), which manages transactions, security, concurrency, and pooling.

There are **3 types** of EJBs:

Type	Subtypes / Note
Session bean	Stateless , Stateful , Singleton
Entity bean	Persistent data; deprecated in EJB 3.0+, replaced by JPA
Message-Driven bean (MDB)	Asynchronous , listens to JMS messages

MCQ Trap: A **JavaBean** is a lightweight, single-JVM, client-side reusable component; an **EJB** is a heavyweight, **server-side**, container-managed, distributed component. They are NOT the same despite the shared word "Bean."

7. Server-Side Programming

7.1 ASP — Active Server Pages

ASP (Active Server Pages) is **Microsoft's** server-side scripting environment. It runs on the **IIS** web server, uses **VBScript** (default) or **JScript**, and files use the **.asp** extension. The server processes the script and returns plain **HTML**. Its modern successor is **ASP.NET** (**.aspx**, **.NET** framework).

ASP built-in objects include **Request**, **Response**, **Session**, **Application**, **Server**, and **ObjectContext**.

7.2 JSP — Java Server Pages

7.2.1 Introduction and Architecture

JSP (Java Server Pages) is a **server-side** technology that lets you embed **Java** code inside HTML using special tags. A JSP page is **translated into a Servlet**, then compiled and executed by the container (e.g., **Apache Tomcat**). It is platform-independent and simplifies dynamic-page creation versus raw servlets.

7.2.2 JSP Lifecycle

```

.jsp file
|
(1) TRANSLATION    -> generates .java Servlet source
|
(2) COMPILATION    -> compiles to .class
|
(3) CLASS LOADING & INSTANTIATION
|
(4) jspInit()      <- called ONCE
|
(5) _jspService()  <- called PER request
|
(6) jspDestroy()   <- called ONCE at end

```

- `jspInit()` and `jspDestroy()` **can** be overridden.
- `_jspService()` is generated automatically and **cannot** be overridden.

MCQ Trap: A JSP is ultimately a **servlet**. The very first request is slower because of **translation + compilation**; later requests reuse the compiled class. `_jspService()` cannot be overridden.

7.2.3 JSP Elements (Scripting + Directives + Actions)

Element	Syntax	Purpose
Scriptlet	<code><% code %></code>	Java code in <code>_jspService</code>
Expression	<code><%= expr %></code>	Output a value (no semicolon)
Declaration	<code><%! decl %></code>	Declare variables/methods
Directive	<code><%@ ... %></code>	page / include / taglib
Action	<code><jsp:... ></code>	useBean, include, forward
Comment	<code><!-- ... --></code>	JSP comment (not sent to client)

- Directives: **page** (import, contentType, session, errorPage, isErrorPage, buffer), **include** (static include at translation time), **taglib**.

7.2.4 JSP Implicit Objects

JSP provides **9 implicit objects** (auto-available in scriptlets/expressions):

Object	Type
request	HttpServletRequest
response	HttpServletResponse
out	JspWriter
session	HttpSession
application	ServletContext
config	ServletConfig
pageContext	PageContext
page	Object (this)

Object	Type
exception	Throwable (error pages only)

MCQ Trap: There are **9** JSP implicit objects. **exception** is available **only** on a page with `isErrorPage="true"`. `out` is a **JspWriter**, not a `PrintWriter`.

7.2.5 Scopes, Conditional Processing, and Data Sharing

- **4 scopes:** **page**, **request**, **session**, **application** (narrowest → widest).
- **Conditional processing:** via scriptlet `if/else/switch`, or JSTL `<c:if>/<c:choose>`.
- **Declaring variables/methods:** instance-level via the declaration tag `<%! ... %>`; local via scriptlets.
- **Sharing data between JSP pages:** through `pageContext`, **request** attributes (forward), **session**, and **application**; or `<jsp:useBean>` with a `scope` attribute.
- **JDBC** can be used directly in a JSP for **database programming**, and **JavaBeans** can be instantiated with `<jsp:useBean>` for clean separation of logic.

8. Servlets

8.1 Introduction

A **Servlet** is a **server-side Java class** that handles client requests and generates **dynamic responses**. It runs inside a **servlet/web container** (e.g., **Tomcat**), is platform-independent, and is more efficient than CGI because it uses **threads**, not processes (one instance serves many requests via multiple threads).

8.2 Servlet Packages and API

- **javax.servlet** — generic, protocol-independent servlets (**GenericServlet**).
- **javax.servlet.http** — HTTP-specific servlets (**HttpServlet**).
- Key interfaces/classes: **Servlet**, **GenericServlet**, **HttpServlet**, **ServletRequest**, **ServletResponse**, **ServletConfig**, **ServletContext**.
- **JSDK (Java Servlet Development Kit)** provides the tools/classes to develop servlets. The `web.xml` file is the **deployment descriptor**.

MCQ Trap: In modern **Jakarta EE 9+** the package was renamed from **javax.servlet** to **jakarta.servlet**, but classic exams still test **javax.servlet** and **javax.servlet.http**.

8.3 Servlet Lifecycle

A servlet has **3 lifecycle methods**:

```

Load & Instantiate
|
init()          <- called ONCE (on load / first request)
|
service()       <- called for EVERY request
|               (dispatches to doGet / doPost / ...)
|
destroy()       <- called ONCE (on container shutdown)

```

- **init()** — runs **once**, for one-time setup.
- **service()** — runs **per request**; inspects the HTTP method and calls `doGet()`, `doPost()`, etc.
- **destroy()** — runs **once** before the servlet is unloaded.

MCQ Trap: `init()` and `destroy()` run **exactly once**; `service()` runs **on every request**. The container creates only **one instance** and uses **multiple threads** — so instance variables are NOT thread-safe.

8.4 Request Handling and Session Tracking

- `doGet()` handles GET requests; `doPost()` handles POST requests (overridden in `HttpServlet`).
- Because HTTP is **stateless**, servlets use **4 session-tracking techniques**:

Technique	How it works
Cookies	Small data stored on the client
URL Rewriting	Append session id to URL
Hidden Form Fields	Store data in hidden inputs
HttpSession	Server-side session object (<code>request.getSession()</code>)

8.5 Servlets vs JSP

Aspect	Servlet	JSP
Code style	Java with HTML embedded	HTML with Java embedded
Best for	Business logic / control	Presentation / view
Compilation	Written/compiled by developer	Auto-translated to a servlet
Ease	Harder (lots of <code>out.println</code>)	Easier for designers

MCQ Trap: A **JSP is converted into a servlet** at runtime, so internally they are the same. Use **servlets** for logic and **JSP** for presentation (MVC pattern).

8.6 JDBC (Database Programming)

JDBC (Java Database Connectivity) is the Java **API** for connecting to relational databases (`java.sql`, `javax.sql`). There are **4 driver types**:

Type	Name	Note
Type 1	JDBC-ODBC bridge	Uses ODBC; now removed
Type 2	Native-API	Partly Java + native libs
Type 3	Network-Protocol	Middleware server
Type 4	Thin driver (pure Java)	Most used today

The **6 JDBC steps**: **(1)** load/register the driver (`Class.forName`), **(2)** open a connection (`DriverManager.getConnection`), **(3)** create a statement, **(4)** execute the query (`executeQuery` for SELECT, `executeUpdate` for INSERT/UPDATE/DELETE), **(5)** process the `ResultSet`, **(6)** close.

MCQ Trap: **Type 1** is the **JDBC-ODBC bridge**; **Type 4** is the **pure-Java thin driver**. Use a **PreparedStatement** (precompiled, parameterized) to prevent **SQL injection**.

9. Distributed Component Technologies

9.1 COM (Component Object Model)

COM is **Microsoft's binary interface standard** that lets software components communicate regardless of the language they were written in (**language-independent**), enabling reuse and inter-process communication. The base interface every COM object implements is **IUnknown** (with `QueryInterface`, `AddRef`, `Release`). It is the foundation of **OLE** and **ActiveX** and is **Windows-only**.

9.2 DCOM (Distributed COM)

DCOM extends **COM** to work **across a network** (components on different machines). It uses **RPC (Remote Procedure Call)** — specifically Object RPC — to marshal calls between machines. It is often described as **"COM with a longer wire."**

9.3 CORBA

CORBA (Common Object Request Broker Architecture) is an **open standard** by the **OMG (Object Management Group)** for distributed objects that is **language- and platform-independent**. Its core is the **ORB (Object Request Broker)**. Interfaces are defined in **IDL (Interface Definition Language)**, and communication uses **IIOP (Internet Inter-ORB Protocol)**. The client uses a **stub**; the server uses a **skeleton**.

9.4 Comparison of COM / DCOM / CORBA

Feature	COM / DCOM	CORBA
Vendor	Microsoft	OMG (open standard)
Platform	Windows	Cross-platform
Interface language	MIDL	IDL
Network protocol	RPC (DCOM)	IIOP
Broker	COM runtime	ORB

MCQ Trap: **COM/DCOM** are **Microsoft/Windows** technologies; **CORBA** is an **OMG**, cross-platform, language-neutral standard using an **ORB** and **IIOP**. DCOM = COM + networking.

10. PHP — Hypertext Preprocessor

10.1 Introduction and Setup

PHP stands for "**PHP: Hypertext Preprocessor**" (a **recursive acronym**; originally **Personal Home Page**). It was created by **Rasmus Lerdorf** in **1994**. PHP is a **server-side, interpreted, open-source, loosely typed** scripting language **embedded in HTML**; files use the **.php** extension. A common stack is **LAMP/WAMP** (Linux/Windows + **Apache** + **MySQL** + **PHP**).

10.2 Syntax

- PHP code lives inside `<?php ... ?>` tags.
- Statements end with a **semicolon ;**.
- Comments: `//` and `#` (single line), `/* ... */` (multi-line).
- Output: **echo** (no return value, multiple args) and **print** (returns **1**, single arg).

```
<?php
    echo "Hello, World!";
    // single-line comment
?>
```

10.3 Variables

- Variables start with **\$**, are **case-sensitive**, **loosely typed**, and need **no declaration**.
- **Variable variables:** `$$name` uses the value of one variable as the name of another.
- **Constants:** defined with `define()` or `const` and use **no \$**.
- Scope: **local**, **global** (`global` keyword / `$GLOBALS`), **static**.
- Data types: **String**, **Integer**, **Float**, **Boolean**, **Array**, **Object**, **NULL**, **Resource**.

10.4 Strings and String Functions

- Single quotes `' '` are **literal**; double quotes `" "` allow **variable interpolation** and escape sequences.
- Concatenation uses the **dot .** operator (NOT **+**).
- Key functions: `strlen()`, `strpos()`, `str_replace()`, `substr()`, `strtoupper()`, `strtolower()`, `ucfirst()`, `trim()`, `explode()`, `implode()`, `strrev()`.

MCQ Trap: In PHP, strings are joined with the **dot .** operator, not **+**. Double quotes parse variables ("Hi \$name"); single quotes do **not**.

10.5 Operators

Arithmetic (+ - * / % **), assignment (= += -=), comparison (== === != !== <>), logical (&& || ! and or), increment/decrement (++ --), the spaceship <=> (PHP 7), and null-coalescing ?? (PHP 7).

MCQ Trap: == checks value with type juggling; === (identical) checks **value AND type**. 0 == "abc" is **false** in PHP 8 (changed from older behavior) — but on equality of value+type, always prefer ===.

10.6 Control Structures

- Conditionals: **if**, **if-else**, **elseif**, **switch**.
- Loops: **for**, **while**, **do-while**, and **foreach** (designed for arrays).
- Jumps: **break**, **continue**.

MCQ Trap: **foreach** is the loop built specifically for iterating **arrays** (foreach(\$arr as \$k => \$v)); a **do-while** runs its body **at least once**.

10.7 Arrays

There are **3 types** of PHP arrays.

Type	Keys	Created with
Indexed	Numeric (from 0)	array(1,2) or [1,2]
Associative	Named keys	["name" => "Sam"]
Multidimensional	Arrays of arrays	nested array()

Useful functions: count(), sort(), rsort(), asort(), ksort(), array_push(), array_pop(), array_merge(), in_array(), array_keys(), array_values().

10.8 Functions

Defined with function name(\$args){ ... }. Support **default arguments**, **return** values, pass **by value** or **by reference** (&), and **variadic** (...\$args). Function names are **case-insensitive**.

10.9 Form Handling (GET/POST)

PHP reads submitted form data through **superglobals**: \$_GET (URL data), \$_POST (body data), \$_REQUEST (both). The form's **method** attribute decides which one is populated.

The **9 PHP superglobals**: \$_GET, \$_POST, \$_REQUEST, \$_SESSION, \$_COOKIE, \$_FILES, \$_SERVER, \$_ENV, \$GLOBALS.

MCQ Trap: Superglobals are **always accessible** in any scope without **global**. File-upload data comes through \$_FILES, not \$_POST.

10.10 Mail

The mail(\$to, \$subject, \$message, \$headers) function sends email from PHP (requires a configured mail server/SMTP).

10.11 File Upload and File Handling

- File upload needs **method="POST"** and **enctype="multipart/form-data"**; the file arrives in \$_FILES with keys **name**, **tmp_name**, **size**, **type**, **error**. Use move_uploaded_file() to save it.
- File handling: fopen(), fread(), fwrite(), fclose(), fgets(), file_get_contents(), file_put_contents(). Modes: **r** (read), **w** (write/truncate), **a** (append), plus **r+**, **w+**, **a+**.

MCQ Trap: File upload **requires** `enctype="multipart/form-data"` — without it `$_FILES` is empty. Mode **w** truncates the file; mode **a** appends.

10.12 Sessions and Cookies

- Sessions** store data **on the server**; start with `session_start()` (before any output), access via `$_SESSION`, end with `session_destroy()`. The session id travels in a cookie named **PHPSESSID** (or in the URL).
- Cookies** store data **on the client**; set with `setcookie(name, value, expire, path)` and read via `$_COOKIE`.

Feature	Session	Cookie
Stored on	Server	Client (browser)
Capacity	Large	~4 KB
Security	More secure	Less secure
Lifetime	Until close/timeout	Set by expire

MCQ Trap: **Sessions are server-side**, **cookies are client-side**. `session_start()` must be called **before any HTML output**, or you get a "headers already sent" error.

10.13 Error and Exception Handling

- Error handling:** `error_reporting()`, `ini_set()`, levels **E_ERROR**, **E_WARNING**, **E_NOTICE**, **E_PARSE**, **E_ALL**; custom handler via `set_error_handler()`; stop with `die()/exit()`.
- Exception handling:** structured with **try**, **catch**, **throw**, **finally** and the **Exception** class.

MCQ Trap: A **finally** block always runs whether or not an exception is thrown. **throw** raises an exception; **catch** handles it.

10.14 Filter (Validation and Sanitization)

PHP filters validate or sanitize data via `filter_var()` and `filter_input()`.

- Validate:** **FILTER_VALIDATE_EMAIL**, **FILTER_VALIDATE_INT**, **FILTER_VALIDATE_URL**.
- Sanitize:** **FILTER_SANITIZE_EMAIL**, **FILTER_SANITIZE_STRING**, **FILTER_SANITIZE_URL**.

MCQ Trap: **Validate** filters **check** whether data is in a valid format (return value or false); **Sanitize** filters **clean/strip** unwanted characters. Don't confuse the two families.

10.15 PHP-ODBC and PHP-MySQL

- PHP-ODBC** connects to any **ODBC**-compliant database: `odbc_connect($dsn, $user, $pass)`, `odbc_exec()`, `odbc_fetch_row()`, `odbc_result()`, `odbc_close()`.
- PHP-MySQL** uses `mysqli` (procedural + OO) or **PDO (PHP Data Objects)**; PDO supports **many** databases while `mysqli` is MySQL-only. The old `mysql_*` extension was **removed in PHP 7**.

MCQ Trap: **ODBC** = Open Database Connectivity, a generic bridge to any compliant DB via a **DSN**. **PDO** supports **multiple** databases; **mysqli** is **MySQL-only**. Always prefer **prepared statements** to stop SQL injection.

11. Quick Revision — Exam Pointers

- HTML form default method is **GET**; use **POST** for sensitive/large data and file uploads.
- CSS specificity: **inline (1000) > id (100) > class (10) > element (1)**; **!important** beats all.
- DOM** = tree/in-memory/random-access/modifiable; **SAX** = event-based/sequential/low-memory/read-only.

- **Well-formed** XML obeys syntax rules; **valid** XML also conforms to a **DTD/XSD**.
- **DTD** has no data types and is not written in XML; **XSD** supports data types + namespaces and IS written in XML.
- JSP has **9 implicit objects** and the lifecycle **jspInit** → **_jspService** → **jspDestroy**.
- Servlet lifecycle = **init()** → **service()** → **destroy()**; only **service()** repeats.
- Session tracking (4): **cookies, URL rewriting, hidden form fields, HttpSession**.
- PHP superglobals: **\$_GET, \$_POST, \$_REQUEST, \$_SESSION, \$_COOKIE, \$_FILES, \$_SERVER, \$_ENV, \$GLOBALS**.
- Client-side (**JavaScript/VBScript**) runs in the browser; server-side (**PHP/JSP/ASP/Servlets**) runs on the server.